CPTS 360 - C Primer

Notes rewritten from course material by Professor Monowar Hasan, Washington State University. AI assistance: OpenAI GPT-5.6 Sol.

1. Uses of C

Main idea: C is useful when software needs close control over memory or hardware.

- Common uses: operating-system kernels, device drivers, debuggers, and embedded/IoT systems.
- C is also used for compilers and interpreters.

2. Data Types

Main idea: A type controls a value's size, memory representation, and legal operations.

- Type sizes can vary between systems.
- Use **sizeof** instead of assuming a platform-specific size.
- Classic C uses zero for false and nonzero for true.
- C99 provides bool, true, and false through **<stdbool.h>**.

```
#include <stdbool.h>
bool ready = true;
printf("%zu\n", sizeof(int));
```

3. Structs

Main idea: A struct groups related values into one custom type.

- Values inside a struct are called members.
- Use **.** to access a member.
- Use **typedef** to create a shorter type name.

```
typedef struct {
    int x, y;
} Point;

Point p;
p.x = 4;
```

4. Pointers

Main idea: A pointer stores a memory address.

- Use **&** to get an address and ***** to dereference a pointer.
- The pointer type describes the value stored at that address.
- NULL represents a pointer that does not identify a valid object.
- Attach * to every pointer variable in a multi-variable declaration.

```
int value = 1;
int *ptr = &value;
printf("%d\n", *ptr);   // 1

int *a, *b, *c;          // all pointers
```

5. Pointers in Functions

Main idea: Passing a pointer lets a function modify the caller's original value.

- Normal parameters are passed by value.
- Struct pointers avoid copying the whole struct and allow changes to it.

- Use **ptr->member** instead of (*ptr).member.

```
void increment(int *n) {
    *n = *n + 1;
}

int value = 0;
increment(&value);        // value is now 1
```

6. void Pointers

Main idea: A void pointer can store an address without specifying the data type.

- A void pointer cannot be dereferenced directly.
- Cast it to the correct pointer type before dereferencing.
- Generic functions such as qsort use void pointers to handle many array types.
- malloc returns a generic pointer to a block of memory.

```
int x = 4;
void *generic = &x;
int value = *(int *)generic;
```

7. Arrays

Main idea: An array stores same-type elements next to one another in memory.

- Total size = sizeof(element) x number of elements.
- The array name acts as the address of the first element.
- array[i] is equivalent to *(array + i).
- Pointer arithmetic advances by elements, not raw bytes.

```
int numbers[32];
int a = numbers[8];
int b = *(numbers + 8);   // same value
```

8. Program Memory

Main idea: Stack, heap, and global data have different purposes and lifetimes.

- Stack: local variables and function parameters.
- Heap: memory allocated at run time.
- Global data: values loaded from the program binary.
- Do not return the address of a local stack variable.

```
int *wrong(void) {
    int local = 5;
    return &local;         // local expires on return
}
```

9. Dynamic Memory

Main idea: malloc requests heap memory and free releases it.

- malloc accepts a size in bytes and returns a pointer.
- Use sizeof to request enough memory for the object.
- Heap memory can remain valid after the allocating function returns.
- Call free when the allocation is no longer needed.
- Setting the freed pointer to NULL can prevent accidental reuse.

```
int *value = (int *)malloc(sizeof(int));
*value = 1;
free(value);
value = NULL;
```

10. Command-Line Arguments

Main idea: argc and argv provide input when a program starts.

- argc is the number of arguments.
- argv is an array of argument strings.
- argv[0] contains the program name; user input begins at argv[1].
- The course example uses atoi to convert strings to integers.

```
int main(int argc, char *argv[]) {
    for (int i = 1; i < argc; i++)
        printf("%d\n", atoi(argv[i]));
    return 0;
}
```

11. Makefiles

Main idea: A Makefile records build commands and file dependencies.

- Target: the file or action to create or update.
- Dependencies: files or targets required by the target.
- Rule: the command that builds the target.
- Macros such as CC, CFLAGS, and OBJS reduce repetition.
- Common named targets include build, run, all, and clean.

```
CC = gcc
CFLAGS = -Wall
OBJS = main.o mysum.o

mytype: type.h $(OBJS)
■$(CC) $(CFLAGS) -o mytype $(OBJS)

clean:
■rm mytype main.o mysum.o
```

Resources from the course material: cdecl.org; C WikiBook; Essential C by Nick Parlante; GNU C Reference Manual.